

Elden Ring Combat Time Optimization: A Resource-Constrained Weapon Selection Problem with State-Dependent Scaling

Kamil Sas 284214

June 6, 2026

1 Introduction

This document presents the challenge of optimizing combat efficiency in the Elden Ring boss-run. The problem is formulated as a **Resource-Constrained Weapon Selection Problem (RCWSP)** with non-linear, state-dependent damage scaling. The primary objective of this framework is to minimize the total active combat time required to defeat a specific sequence of bosses by determining the optimal synergy between discrete weapon selection and quasi-continuous attribute resource allocation, subject to dynamic capacity constraints.

The formulation establishes a functional relationship between character endurance and weapon execution speed. By mapping the unallocated attribute point pool alongside a fixed baseline capacity directly to weapon swing frames, the model captures a strategic trade-off between raw scaling damage per hit and high-velocity attack recovery.

2 Formal Mathematical Definition

The optimization framework is formally defined by the tuple:

$$P = (I, J, D, M, W, T^{\text{base}}, HP, R, s, e, B)$$

2.1 Sets and Parameters

- $I = \{1, 2, \dots, n\}$: The finite set of candidate weapons available.
- $J = \{1, 2, \dots, m\}$: The finite set of bosses included in the run gauntlet.
- $D = \{1, 2, \dots, d\}$: The finite set of distinct damage categories, defined as:

$$D = \{\text{Standard, Strike, Slash, Piercing, Magic, Fire, Lightning, Holy}\}$$

- $M \in \mathbb{R}_+^{n \times d}$: A coefficient matrix where the element $M_{i,k}$ represents the baseline offensive scaling multiplier of the weapon i corresponding to the damage type k .
- $W = (W_1, W_2, \dots, W_n) \in \mathbb{R}_+^n$: A vector representing the equipment weight profile of each individual weapon.
- $T^{\text{base}} = (T_1^{\text{base}}, T_2^{\text{base}}, \dots, T_n^{\text{base}}) \in \mathbb{R}_+^n$: A vector containing the base animation execution time (in seconds) required for a single swing of the weapon i .

- $HP = (HP_1, HP_2, \dots, HP_m) \in \mathbb{Z}_+^m$: A vector containing the raw health pool values for each boss $j \in J$.
- $R \in [0, 1]^{m \times d}$: A resistance matrix where $R_{j,k}$ denotes the damage mitigation factor of boss j against damage type k .
- $s \in \mathbb{Z}_+$: The total pool of attribute points available for allocation ($s = 100$).
- $e \in \mathbb{Z}_+$: The baseline endurance capacity constant ($e = 10$).
- $B \in \mathbb{R}_+$: A sufficiently large positive scalar constant used for logical constraint enforcement ($B \gg 0$).

2.2 Decision Variables

- $x \in \mathbb{Z}_+$: An integer allocation variable indicating the investment of points in the primary offensive attribute ($1 \leq x \leq s$).
- $w \in \mathbb{R}_+$: A continuous substitution variable mapping the non-linear transformation representing the square root of allocated offensive stats ($1.0 \leq w \leq \sqrt{s}$).
- $y = (y_1, y_2, \dots, y_n) \in \{0, 1\}^n$: A binary selection vector indicating whether the weapon i is actively chosen ($y_i = 1$) or not ($y_i = 0$).

2.3 Mathematical Model Formulation

Let $dmg(w, i, j)$ denote the net damage dealt per physical strike by weapon i to boss j :

$$dmg(w, i, j) = \sum_{k \in D} (w \cdot M_{i,k}) \cdot (1 - R_{j,k})$$

Let $\tau(x, i)$ represent the dynamic, scaling swing time of weapon i , which decreases linearly as the available endurance pool ($s - x + e$) increases:

$$\tau(x, i) = T_i^{\text{base}} \cdot \left(1.0 - 0.5 \cdot \frac{s - x + e}{100} \right)$$

Let $h_j(w, y)$ define the derived functional mathematical expression representing the total number of hits required to defeat boss j based on the weapon choice profile y :

$$h_j(w, y) = \left\lceil \frac{HP_j}{\sum_{i \in I} y_i \cdot dmg(w, i, j)} \right\rceil$$

The overarching formal optimization model is structured as follows:

$$\text{Minimize } T(x, w, y) = \sum_{j \in J} \left(h_j(w, y) \cdot \sum_{i \in I} y_i \cdot \tau(x, i) \right) \quad (1)$$

subject to

$$x \leq s \tag{2}$$

$$w^2 = x \tag{3}$$

$$\sum_{i \in I} y_i = 1 \tag{4}$$

$$\sum_{i \in I} y_i \cdot W_i + x \leq s + e \tag{5}$$

$$\text{dmg}(w, i, j) \geq 1 - B \cdot (1 - y_i) \quad \forall i \in I, \forall j \in J \tag{6}$$

$$x \in \mathbb{Z}_+, \quad w \in \mathbb{R}_+, \quad y \in \{0, 1\}^n$$

2.3.1 Constraint Descriptions

- **Equation (2) (Resource Allocation Bound):** Constrains the maximum investment in the offensive attribute to the total available baseline attribute pool s .
- **Equation (3) (Square-Root Link Constraint):** Enforces a non-linear mapping between the discrete attribute investment x and the continuous auxiliary variable w , representing the square-root scaling relationship of offensive power.
- **Equation (4) (Weapon Selection Constraint):** Forces the optimization algorithm to select exactly one active weapon from the available set I .
- **Equation (5) (Weight Requirement):** Ensures that the weight of the chosen weapon combined with the allocated offensive stat does not exceed the total dynamic endurance threshold $(s - x + e)$.
- **Equation (6) (Weapon Damage Lower Bounding):** Conditional constraint ensuring that if the weapon i is actively selected, its calculated net damage output against the boss j is strictly bounded below by a baseline value of 1.

3 Metaheuristics

To evaluate the objective value $T(x, w, y)$ for any arbitrary solution candidate state defined by weapon index $i \in I$ and stat assignment $x \in \mathbb{Z}_+$, a consolidated cost mapping function $f(i, x)$ is formulated:

$$f(i, x) = \begin{cases} \sum_{j \in J} \left\lceil \frac{HP_j}{\sum_{k \in D} (\sqrt{x} \cdot M_{i,k}) (1 - R_{j,k})} \right\rceil \cdot T_i^{\text{base}} \left(1 - \frac{s-x+e}{200}\right) & \text{if } (s - x + e) \geq W_i \text{ and } \forall j : \text{dmg} \geq 1 \\ \infty & \text{otherwise} \end{cases}$$

Algorithm SA: Simulated Annealing Engine

Require: $I, J, D, W, M, T^{\text{base}}, HP, R, s, e, T_{\text{max}}, T_{\text{min}}, p_{\text{swap}}$

Ensure: $i_{\text{best}}, x_{\text{best}}, f_{\text{best}}$

```
1: Generate initial configuration  $(i, x)$  uniformly at random such that  $f(i, x) < \infty$ .
2: Initialize best configurations:  $(i_{\text{best}}, x_{\text{best}}) \leftarrow (i, x)$  and  $f_{\text{best}} \leftarrow f(i, x)$ .
3: Calculate static total iteration size  $N \leftarrow |I| \cdot s$  and cool factor  $\alpha \leftarrow (T_{\text{min}}/T_{\text{max}})^{1/N}$ .
4: Set initial temperature state  $T \leftarrow T_{\text{max}}$ .
5: for  $k = 1$  to  $N$  do
6:   Sample a random probability parameter  $u \sim \mathcal{U}(0, 1)$ .
7:   if  $u < p_{\text{swap}}$  then
8:     Select a candidate weapon  $i' \in \{w \in I \mid w \neq i\}$  and set  $x' \leftarrow x$ .
9:   else
10:    Set  $i' \leftarrow i$  and sample a step parameter  $\Delta x \in \{-2, -1, 1, 2\}$ .
11:    Compute bounded candidate stat:  $x' \leftarrow \max(1, \min(s, x + \Delta x))$ .
12:   end if
13:   Evaluate candidate objective state value:  $f' \leftarrow f(i', x')$ .
14:   if  $f' < \infty$  then
15:     if  $f' < f(i, x)$  then
16:        $(i, x) \leftarrow (i', x')$ 
17:     else
18:       Sample acceptance scale reference  $v \sim \mathcal{U}(0, 1)$ .
19:       if  $v < \exp\left(-\frac{f' - f(i, x)}{T}\right)$  then
20:          $(i, x) \leftarrow (i', x')$ 
21:       end if
22:     end if
23:     if  $f(i, x) < f_{\text{best}}$  then
24:        $(i_{\text{best}}, x_{\text{best}}) \leftarrow (i, x)$  and  $f_{\text{best}} \leftarrow f(i, x)$ .
25:     end if
26:   end if
27:    $T \leftarrow T \cdot \alpha$ 
28: end for
```

3.1 Complexity Analysis

Based on the algorithm defined in the previous section, the computational requirements are as follows:

Time Complexity: The algorithm executes for a total of $N = |I| \cdot s$ iterations. Since each evaluation of the cost mapping function $f(i, x)$ requires iterating over all bosses J and damage types D , the total time complexity is $O(|I| \cdot s \cdot |J| \cdot |D|)$.

Memory Complexity: The memory usage is dominated by the storage of the weapon and boss data matrices, resulting in a space complexity of $O(|I| \cdot |D| + |J| \cdot |D|)$.

4 Optimization Engine Performance Comparison

Table 1: Comparative Analysis of Optimization Engines

Feature	Gurobi Solver	Simulated Annealing
Time complexity	$O(2^{ I \cdot J } \cdot \log(s) \cdot D)$	$O(I \cdot J \cdot s \cdot D)$
Result Type	Global Optimum	Near-optimal

Table 2: Performance Comparison: Gurobi MIP vs. Simulated Annealing

Dataset	Weapons	Bosses	Gurobi	Simulated Annealing		
			Time (ms)	Time (ms)	Opt. Found	Avg. Error
1	9	6	284.02	7.08	100%	0.00%
2	51	14	3565.36	82.44	100%	0.00%
3	100	50	56638.61	560.67	100%	0.00%
4	200	3	2154.51	100.91	98%	0.00041%
5	300	6	11820.22	303.05	90%	0.0163%

5 Technical Implementation

The optimization models were developed in Python 3.10. The MIP model utilizes the Gurobi Optimizer (v11.0) with non-convex settings to handle the non-linear relationship between attribute investment and combat throughput. The metaheuristic approach is implemented as a custom procedural script utilizing a cooling schedule defined by the parameters derived in the complexity analysis.

Note on Implementation: Running the MIP model across the provided datasets requires a valid Gurobi license (or a free academic/trial license), as the problem size exceeds the constraints of the standard Gurobi limited version.

5.1 Data Structure

The optimization requires two primary input matrices for weapon performance profiles and boss resistance data. The model accepts datasets dynamically, where file inputs follow the convention `weapons{n}.csv` and `bosses{n}.csv` for complexity profiles $n \in \{1, 2, 3, 4, 5\}$. The configuration schema is formatted as follows:

```

1 WeaponName,Weight,Standard,Strike,Slash,Piercing,Magic,Fire,Lightning,Holy,BaseSwingTime
2 Straight_Sword,3.0,100,100,100,100,0,0,0,0,0.85
3 Greatsword,10.0,120,120,120,100,0,0,0,0,1.20
4 ...

```

Listing 1: Example of weapon data structure (e.g., weapons1.csv)

```

1 BossName,HP,Standard,Strike,Slash,Piercing,Magic,Fire,Lightning,Holy
2 Margit_The_Fell,4000,0.1,0.1,0.1,0.1,0.2,0.2,0.2,0.4
3 Rennala_Queen_Full_Moon,4500,0.0,0.0,0.0,0.0,0.8,0.2,0.2,0.2
4 ...

```

Listing 2: Example of boss data structure (e.g., bosses1.csv)

This consistent data schema across all datasets ensures reproducibility, allowing for the direct performance comparison observed in Table 2.